

Unit 1

Basic Concepts and Principles of OOP

1. **Object:** An instance of a class, representing a real-world entity with attributes (data) and behaviors (methods).
2. **Class:** A blueprint or template for creating objects, defining their structure (data) and behaviors (methods).
3. **Encapsulation:** Bundling data and methods into a single unit (class) and restricting direct access to them. This promotes data security and hides internal details from outside interference.
4. **Abstraction:** Hiding complex implementation details and showing only the essential features of an object. This simplifies the interaction with objects and focuses on what an object does rather than how it does it.
5. **Inheritance:** Allows one class (subclass) to inherit the properties and methods of another class (superclass). This enables code reusability and establishes a hierarchy between classes.
6. **Polymorphism:** Allows objects to be treated as instances of their parent class. The same method can have different implementations based on the object calling it, enhancing flexibility and scalability in coding.
7. **Dynamic Binding:** Linking a procedure call to the code to be executed only at runtime. Java uses dynamic binding to achieve runtime polymorphism.
8. **Message Passing:** Objects communicate with one another by sending and receiving information through method calls.

Program Structure in Java

Introduction

Java is an object-oriented, platform-independent, and high-level programming language. It is widely used for building secure, scalable applications. A typical Java program consists of classes and objects, using Java's syntax and structures to perform tasks.

Writing Simple Java Programs

A basic Java program has the following structure:

```
public class HelloWorld {  
  
    public static void main(String[] args) {
```

```
    System.out.println("Hello, World!");  
  
}  
  
}
```

- **Class Declaration:** Each Java program must have at least one class definition, like ``HelloWorld``.

`main` Method: The entry point for the program, where execution begins. It's defined as ``public static void main(String[] args)``.

System.out.println: Used for displaying output on the screen.

Elements or Tokens in Java Programs

1. **Keywords:** Reserved words with special meanings, like ``class``, ``public``, ``static``, ``void``.
2. **Identifiers:** Names given to variables, classes, methods, etc. Must begin with a letter, underscore (`_`), or dollar sign (`$`).
3. **Literals:** Constants used directly in code, such as numbers (``42``), characters (``'A'``), and strings (``"Hello"``).
4. **Operators:** Symbols used for operations, like ``+``, ``-``, ``*``, ``=``.
5. **Separators:** Special symbols like ``{``, ``(``, ``)``, ``;`` used to define blocks, enclose conditions, etc.

Java Statements

Statements are commands that tell the program what to do, ending with a semicolon ``;``. Java statements include:

1. **Declaration Statements:** Define variables, e.g., ``int num = 10;``.
2. **Expression Statements:** Perform calculations or value assignments, e.g., ``num = num + 5;``.
3. **Control Statements:** Manage the flow of the program, including loops (``for``, ``while``) and conditionals (``if``, ``else``).

Command Line Arguments

Command line arguments are inputs passed to the program when it starts. They are stored as an array in the ``main`` method's parameter:

Program:

```
public static void main(String[] args) {  
  
    if (args.length > 0) {  
  
        System.out.println("Argument: " + args[0]);  
  
    }  
  
}
```

To run the above program with a command line argument, use:

```
``bash
```

```
java HelloWorld YourArgument
```

User Input in Java Programs

Java provides various ways to receive user input:

1. **Scanner Class:** A common method for reading input from the console.

Program:

```
import java.util.Scanner;
```

```
public class UserInputExample {  
  
    public static void main(String[] args) {  
  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.print("Enter your name: ");  
  
        String name = scanner.nextLine();  
  
        System.out.println("Hello, " + name);  
  
    }  
  
}
```

2. **BufferedReader:** Reads text from a character input stream, suitable for larger input.

Escape Sequences

Escape sequences are special characters that have specific meanings, beginning with a backslash `\:`

`\n`: Newline

`\t`: Tab

`\"`: Double quote

`\\`: Backslash

Example:

```
System.out.println("Hello\nWorld"); // Outputs "Hello" and "World" on separate lines
```

Comments in Java:

Comments are notes ignored by the compiler, used to explain code:

1. ****Single-line Comment****: Begins with `//`, used for brief explanations.
2. ****Multi-line Comment****: Enclosed between `/\* ... \*/`, used for longer explanations.
3. ****Documentation Comment****: Begins with `/\*\*` and used for JavaDoc.

Programming Style:

Good programming style is essential for readability and maintainability:

1. **Indentation**: Indent code blocks for clarity.
2. **Meaningful Names**: Use descriptive variable, method, and class names.
3. **Consistent Braces**: Use braces `{}` consistently to mark code blocks.
4. **Commenting**: Add comments to explain complex logic.
5. **Modularity**: Break down large programs into methods or classes for better organization.

Data Types, Variables, and Operators

Introduction

Java is a strongly-typed language, meaning each variable must be declared with a data type before it's used. Java provides a variety of data types, variables, and operators for storing, manipulating, and processing data in a structured way.

Data Types in Java

Java has two main categories of data types:

1.: Built-in data types that store simple values.

byte: 8-bit signed integer

short: 16-bit signed integer

int: 32-bit signed integer

long: 64-bit signed integer

float: 32-bit floating point

double: 64-bit floating point

char: 16-bit Unicode character

boolean: Represents `true` or `false`

2. **Non-Primitive (Reference) Data Type**: Refers to objects and can be used to access fields and methods.

String: Sequence of characters.

Arrays*: : Collection of elements of the same data type.

Classes and Objects: Used in object-oriented programming.

Interfaces and Enums: Used to define collections of constants and sets of methods.

Primitive Data Types

Declaration of Variables

Variables in Java are containers for storing data values. A variable declaration includes a data type and a unique name (identifier):

```
int age;    // Declares an integer variable named age
```

```
double salary; // Declares a double variable named salary
```

Data Types and Type Casting

1. **Implicit Casting (Widening)**: Converting a smaller type to a larger type automatically

```
int num = 10;  
  
double result = num; // int to double (widening)
```

2. **Explicit Casting (Narrowing)**: Manually converting a larger type to a smaller type

```
double score = 90.5;  
  
int intScore = (int) score; // double to int (narrowing)
```

Scope of Variables

1. **Local Variables**: Declared inside a method or block, accessible only within that method or block.
2. **Instance Variables**: Declared within a class but outside any method, representing object properties.
3. **Class (Static) Variables**: Declared with the `static` keyword, shared among all instances of the class.

Identifier and Literal Constants

Identifier: The name given to variables, methods, classes, etc. (e.g., `age`, `salary`).

Literal Constants: Fixed values directly assigned to variables (e.g., `10`, `3.14`, `"Hello"`).

Symbolic Constants

Symbolic constants are named constants created using the `final` keyword. Once assigned, they cannot be changed.

```
final int MAX_AGE = 100;
```

Formatted Output with `printf()` Method

Java's `printf()` method allows formatted output, commonly used for numbers, strings, and date-time values.

Example:

```
System.out.printf("Name: %s Age: %d Salary: %.2f", name, age, salary);
```

`%s` for Strings

`%d` for integers

`%.2f` for floating-point numbers with 2 decimal places

Static Variables and Methods

Static Variables: Shared among all instances of a class. Declared with the `static` keyword.

Static Methods: Can be called without creating an object. They can only access static variables and cannot refer to `this` or `super`.

Attribute `final`

The `final` keyword makes variables, methods, or classes immutable.

1. **Final Variables:** Value cannot be changed after initialization.
2. **Final Methods:** Cannot be overridden by subclasses.
3. **Final Classes:** Cannot be subclassed

Introduction to Operators

Operators are special symbols that perform operations on variables and values. Types of operators include:

1. **Arithmetic Operators:** Perform basic mathematical operations.
2. **Relational Operators:** Compare values.
3. **Logical Operators:** Used with Boolean values for logical operations.
4. **Assignment Operators:** Assign values to variables.
5. **Bitwise Operators:** Perform operations on bits.

Precedence and Associativity of Operators

Operator Precedence: Determines the order of operation execution.

Associativity: Determines the direction (left-to-right or right-to-left) in which operators of the same precedence level are evaluated.

Example:

```
int result = 10 + 5 * 2; // result is 20 because * has higher precedence than +
```

Operators in Java

1. **Assignment Operator (=):** Assigns the right-hand value to the left-hand variable.

```
int num = 10;
```

2. **Basic Arithmetic Operators:** Perform arithmetic operations:

`+` (Addition)

`-` (Subtraction)

`\*` (Multiplication)

`/` (Division)

`%` (Modulus - returns remainder)

3. **Increment (`++`) and Decrement (`--`) Operators:** Increase or decrease a variable's value by one.

Postfix: `x++` or `x--` (value changes after usage)

Prefix: `++x` or `--x` (value changes before usage)

4. **Ternary Operator (`?:`):** A shorthand for `if-else` statements.

```
int result = (age > 18) ? "Adult" : "Minor";
```

5. **Relational Operators:** Compare two values and return a boolean result.

`==` (Equal to)

`!=` (Not equal to)

`>` (Greater than)

`<` (Less than)

`>=` (Greater than or equal to)

`<=` (Less than or equal to)

6. **Boolean Logical Operators:** Used in conditional statements with Boolean values.

`&&` (Logical AND)

`||` (Logical OR)

`!` (Logical NOT)

7. **Bitwise Logical Operators:** Operate on individual bits of integer values.

`&` (Bitwise AND)

`|` (Bitwise OR)

`^` (Bitwise XOR)

`~` (Bitwise NOT)

`<<` (Left Shift)

`>>` (Right Shift)

1. Introduction to Control Statements

Control statements are used to control the flow of execution in a program.

They allow the program to make decisions, repeat tasks, or perform a sequence of actions.

The main categories of control statements are:

Conditional Statements: Making decisions based on conditions.

Iteration Statements: Repeating actions multiple times.

Jump Statements: Changing the flow abruptly.

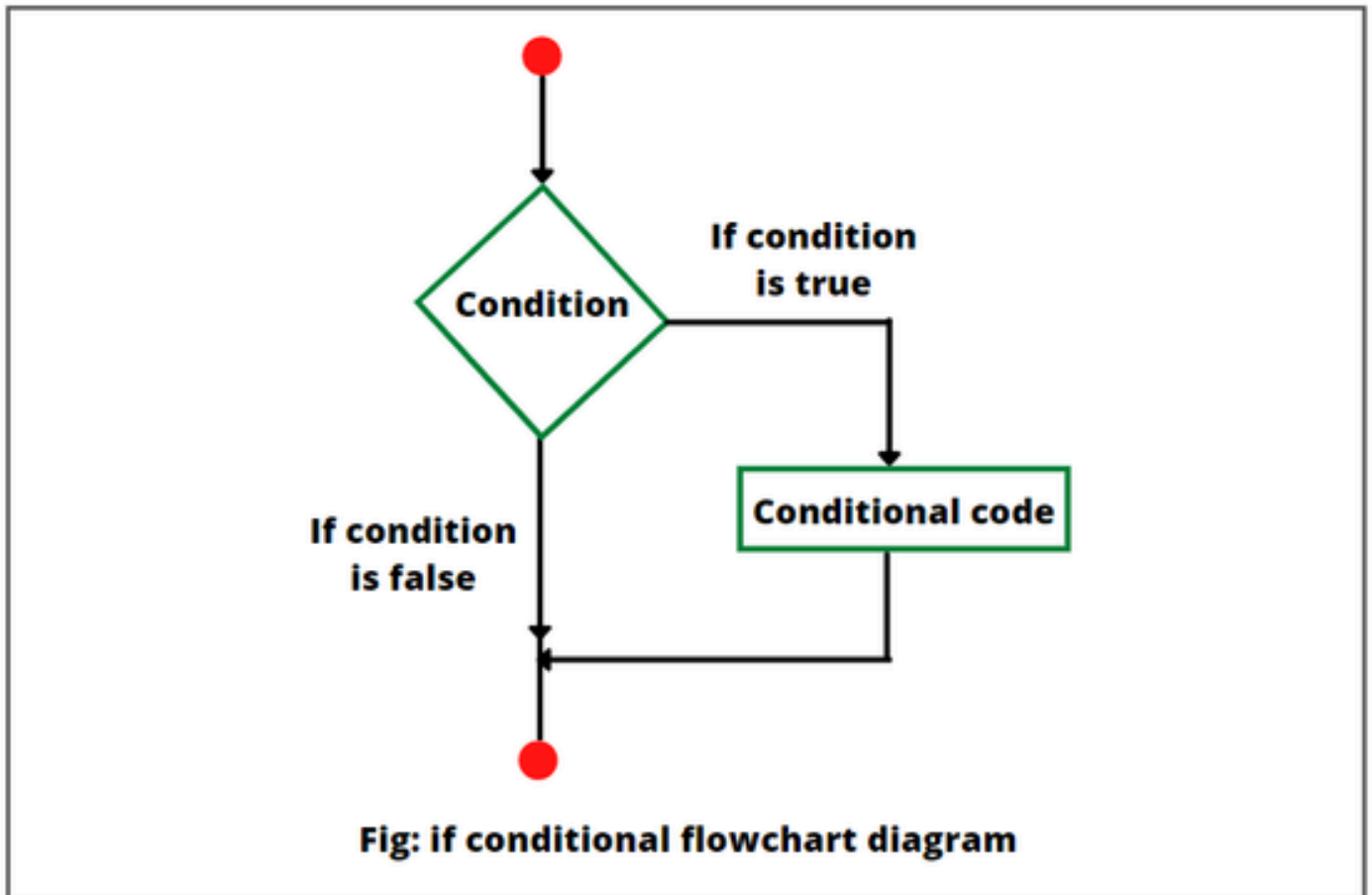
2. Conditional Statements

2.1 **if Expression**

The `if` expression checks a condition and executes the block of code only if the condition is true.

Syntax:

```
java
if (condition) {
    // Code to execute if condition is true
}
```



Example:

```
java
```

```
int age = 18;
```

```
if (age >= 18) {
```

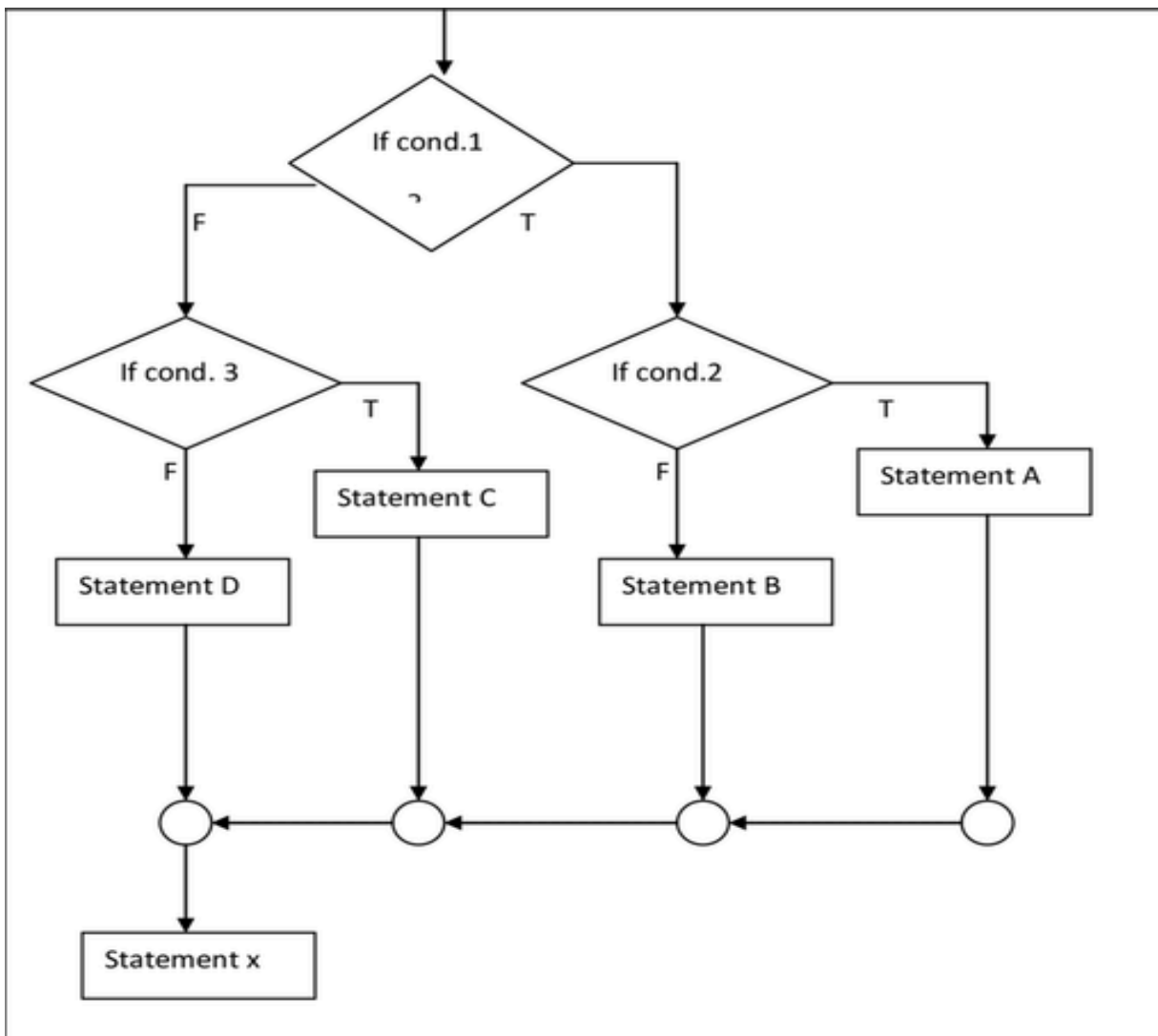
```
    System.out.println("Eligible to vote.");
```

```
}
```

2.2 Nested if Expressions

A nested `if` expression means an `if` statement inside another `if` statement.

Useful for checking multiple conditions.



Example:

```

int age = 20;

int height = 170;

if (age >= 18) {
    if (height >= 160) {
        System.out.println("Eligible for admission.");
    }
}

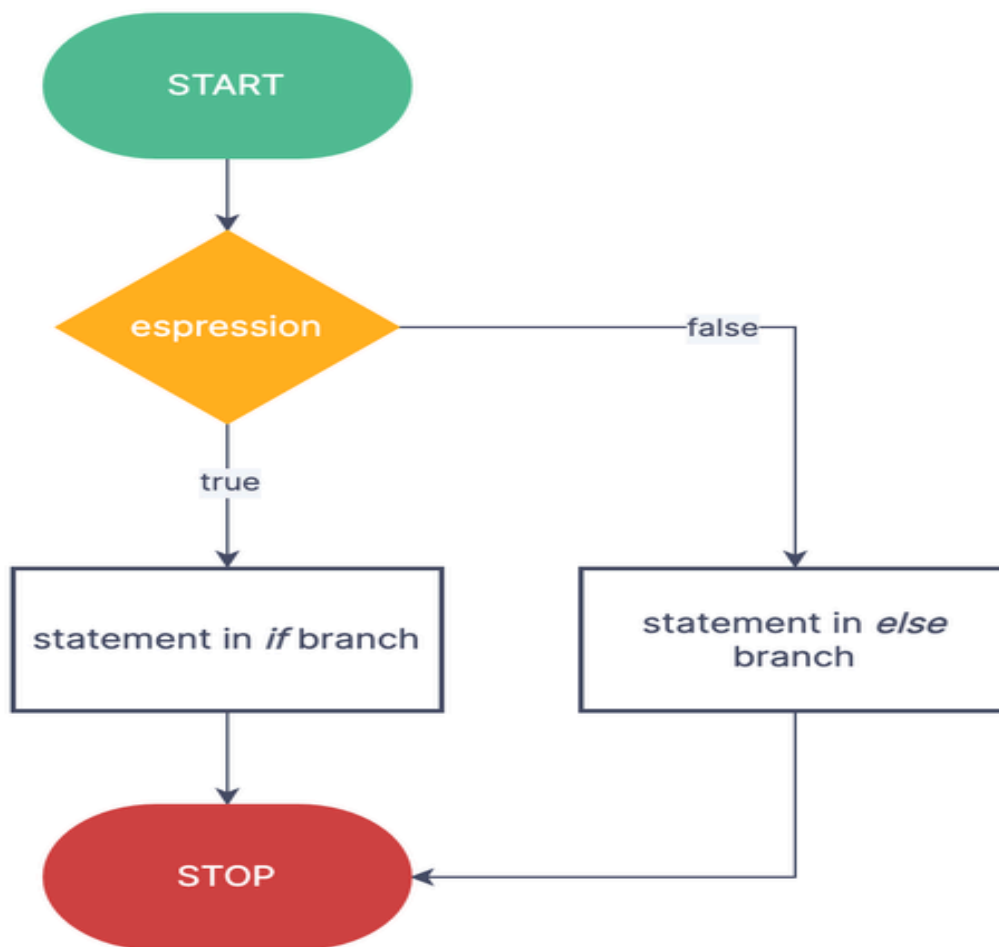
```

2.3 if-else Expressions

The `if-else` expression allows the program to choose between two blocks of code based on a condition.

Syntax:

```
if (condition) {  
    // Code if condition is true  
} else {  
    // Code if condition is false  
}
```



Example:

```
int age = 16;  
  
if (age >= 18) {  
    System.out.println("Eligible to vote.");  
} else {
```

Example:

```
int age = 16;  
  
if (age >= 18) {  
    System.out.println("Eligible to vote.");  
} else {  
    System.out.println("Not eligible to vote.");  
}
```

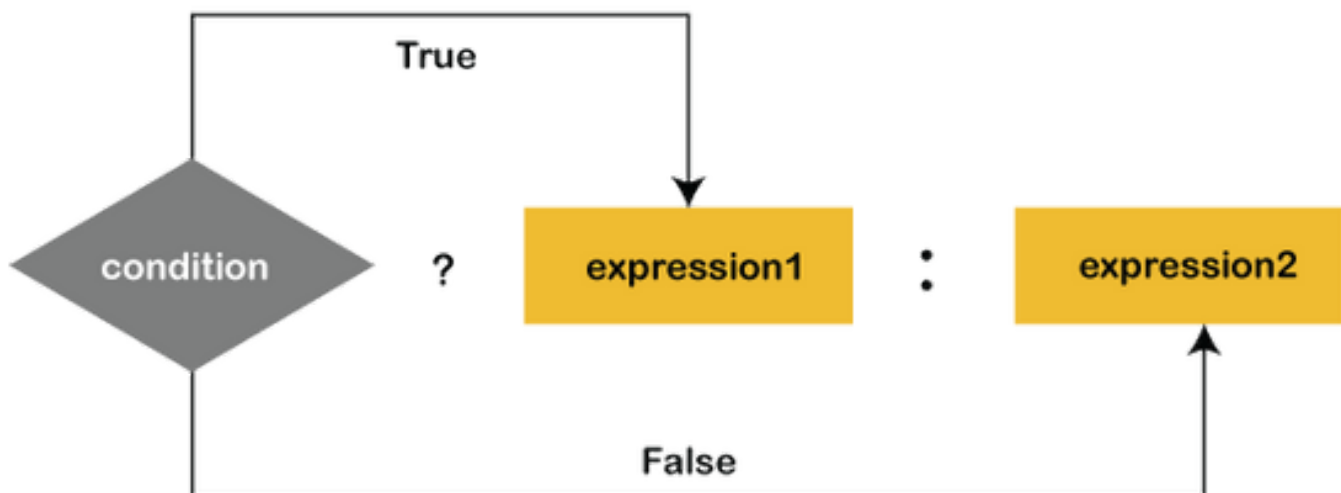
2.4 Ternary Operator(?):

The ternary operator is a shorthand for `if-else` and is written as `condition ? expr1 : expr2`.

Syntax:

```
java
```

```
variable = (condition) ? expression_if_true : expression_if_false;
```

**Example:**

```
int age = 20;
```

```
String result = (age >= 18) ? "Adult" : "Minor";
```

2.5 Switch Statement

The `switch` statement allows multiple conditions to be checked, using a set of cases.

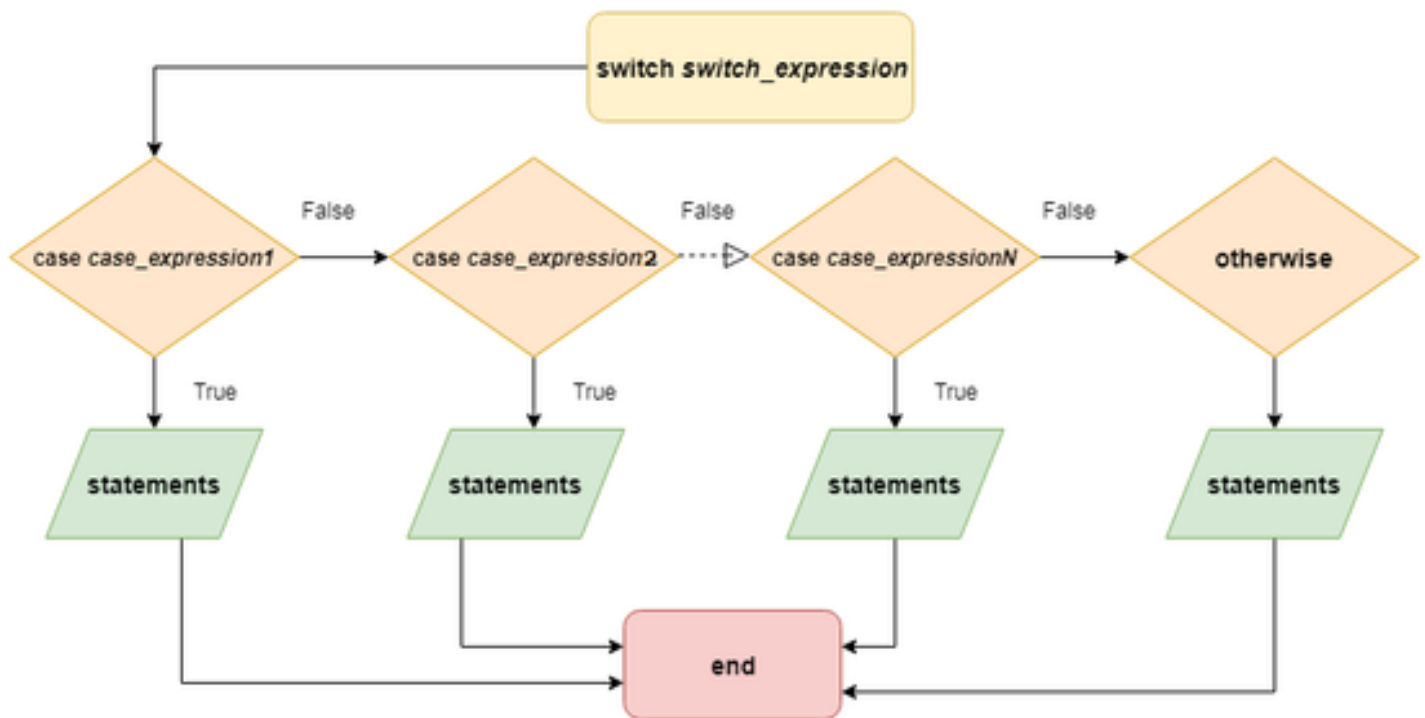
Best used when there are multiple possible discrete values for a variable.

Syntax:

```
switch (variable) {  
    case value1:  
        // Code for value1  
        break;  
    case value2:  
        // Code for value2  
        break;  
    default:  
        // Code if no cases match  
}
```

switch statement flow chart

....switch....case....otherwise....end....



Example:

```
int day = 3;

switch (day) {

    case 1: System.out.println("Monday"); break;

    case 2: System.out.println("Tuesday"); break;

    case 3: System.out.println("Wednesday"); break;

    default: System.out.println("Invalid day");

}
```

3. Iteration Statements

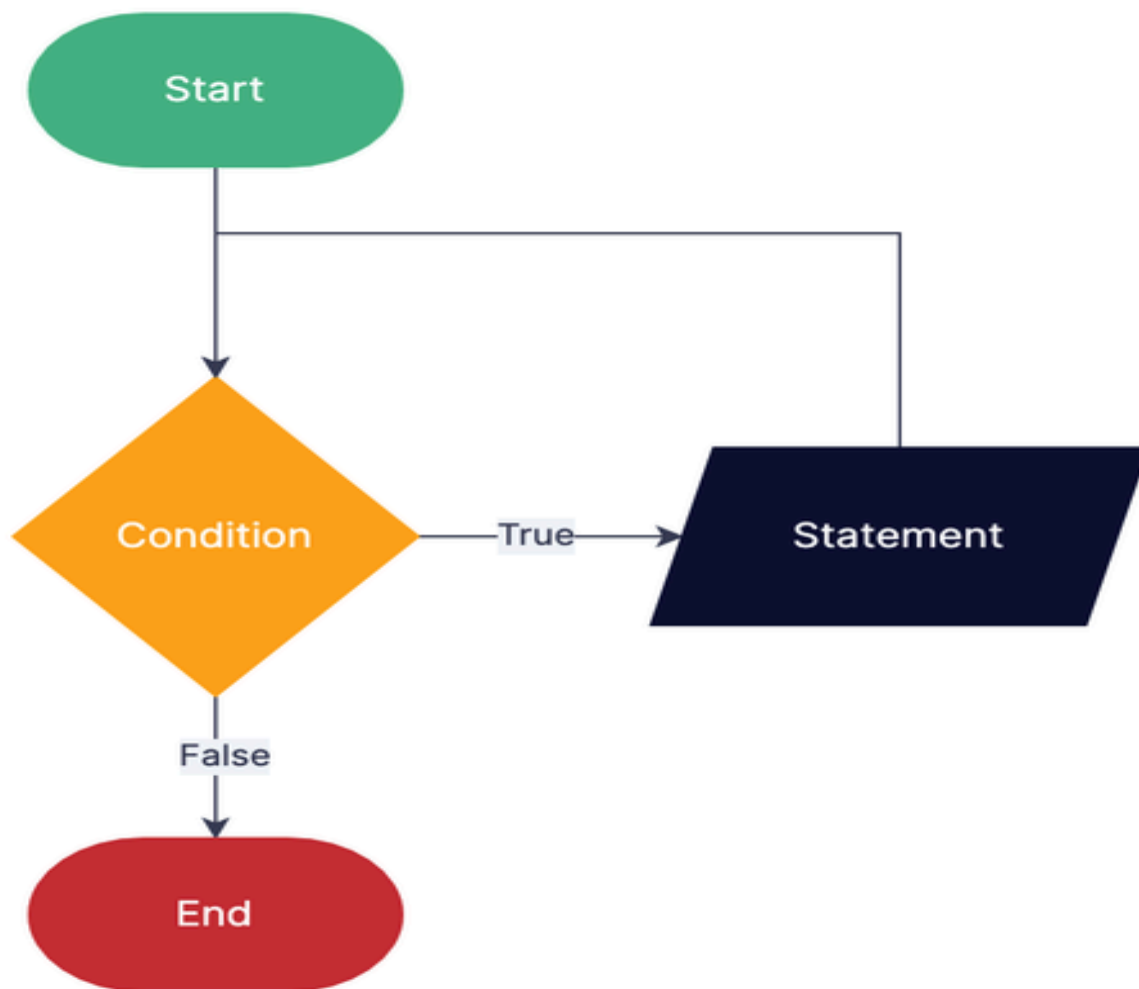
3.1 while Expression

The `while` loop repeats a block of code as long as the condition is `true`.

Syntax:

```
while (condition) {  
    // Code to execute repeatedly  
}
```

While Loop Flowchart

**Example:**

```
int i = 1;  
while (i <= 5) {  
    System.out.println(i);
```

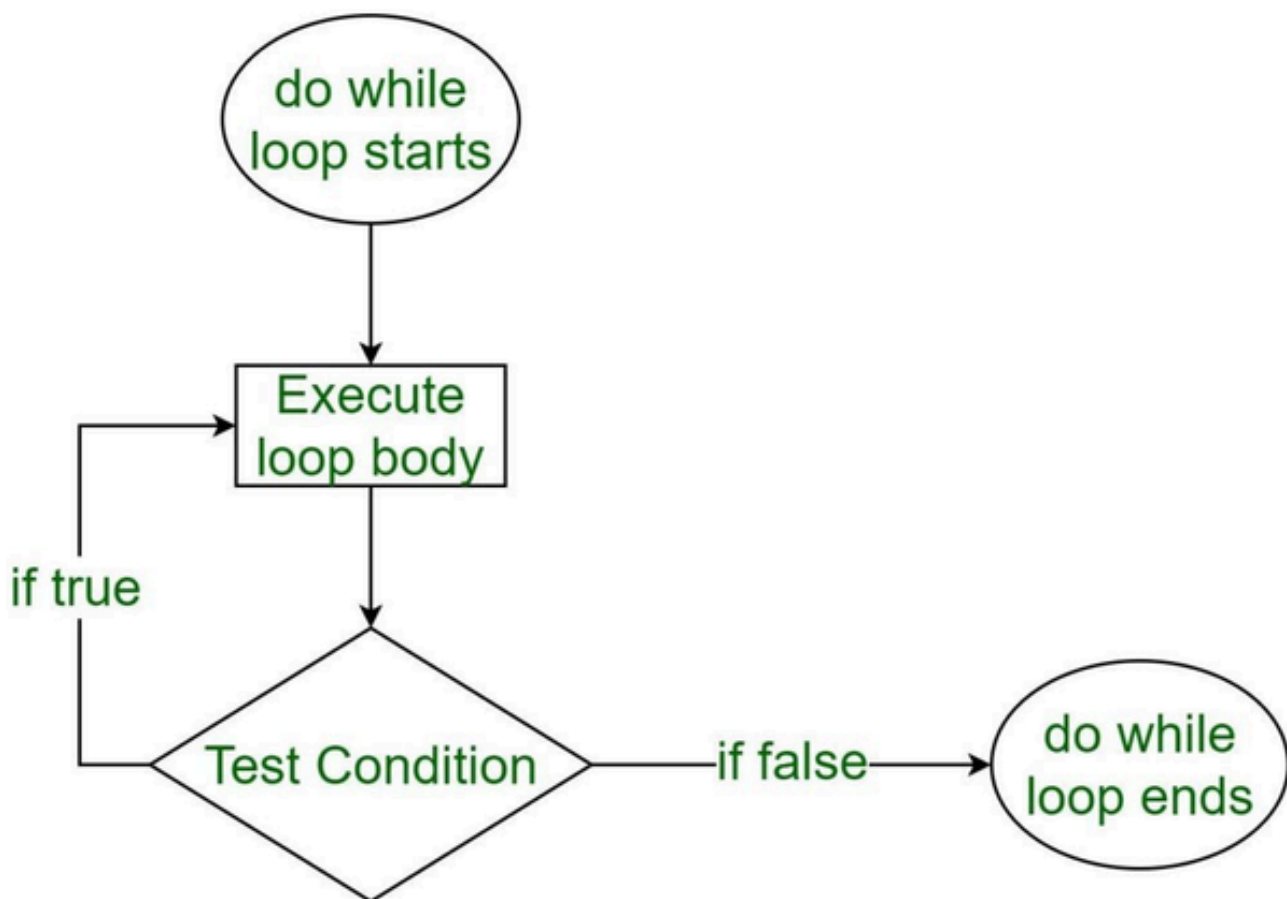
```
i++;  
  
}
```

3.2 do-while Loop

Similar to `while`, but guarantees at least one execution as the condition is checked after the loop.

Syntax:

```
do {  
    // Code to execute  
} while (condition);
```



Example:

```
int i = 1;  
  
do {
```

```
System.out.println(i);  
  
i++;  
  
} while (i <= 5);
```

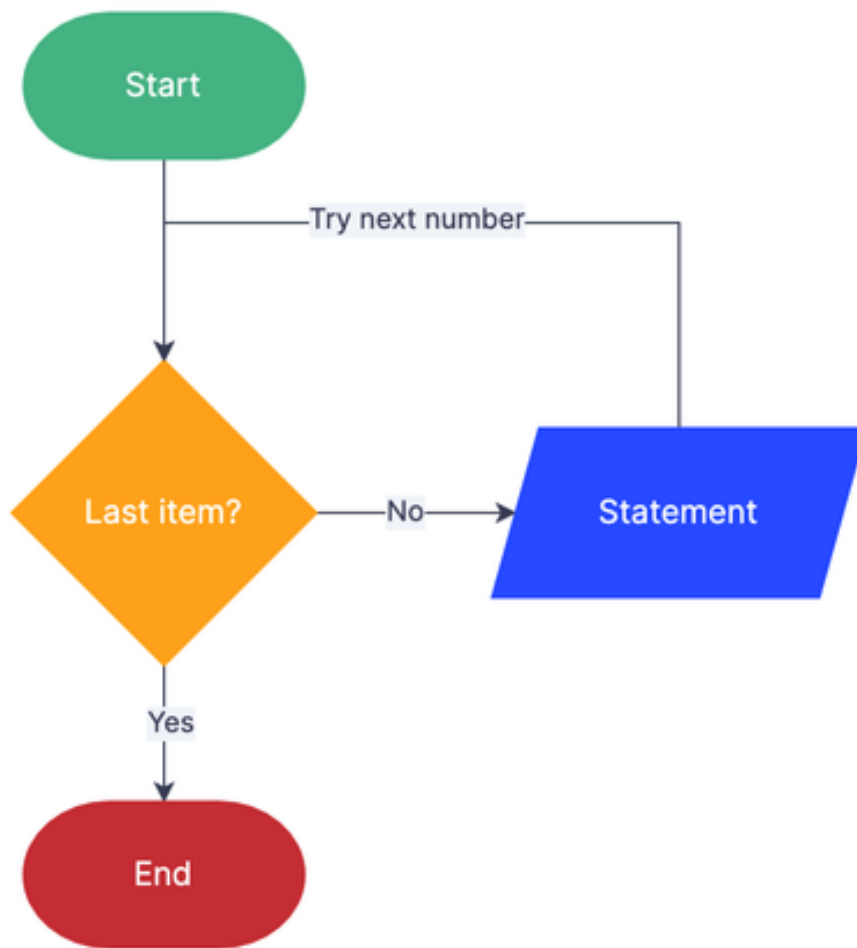
3.3 for Loop

- The `for` loop allows specifying initialization, condition, and increment in one line.

Syntax:

```
for (initialization; condition; increment) {  
  
    // Code to execute  
  
}
```

For Loop Flowchart



Example:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

3.4 Nested for Loop

- A `for` loop inside another `for` loop, commonly used in matrix or tabular structures.

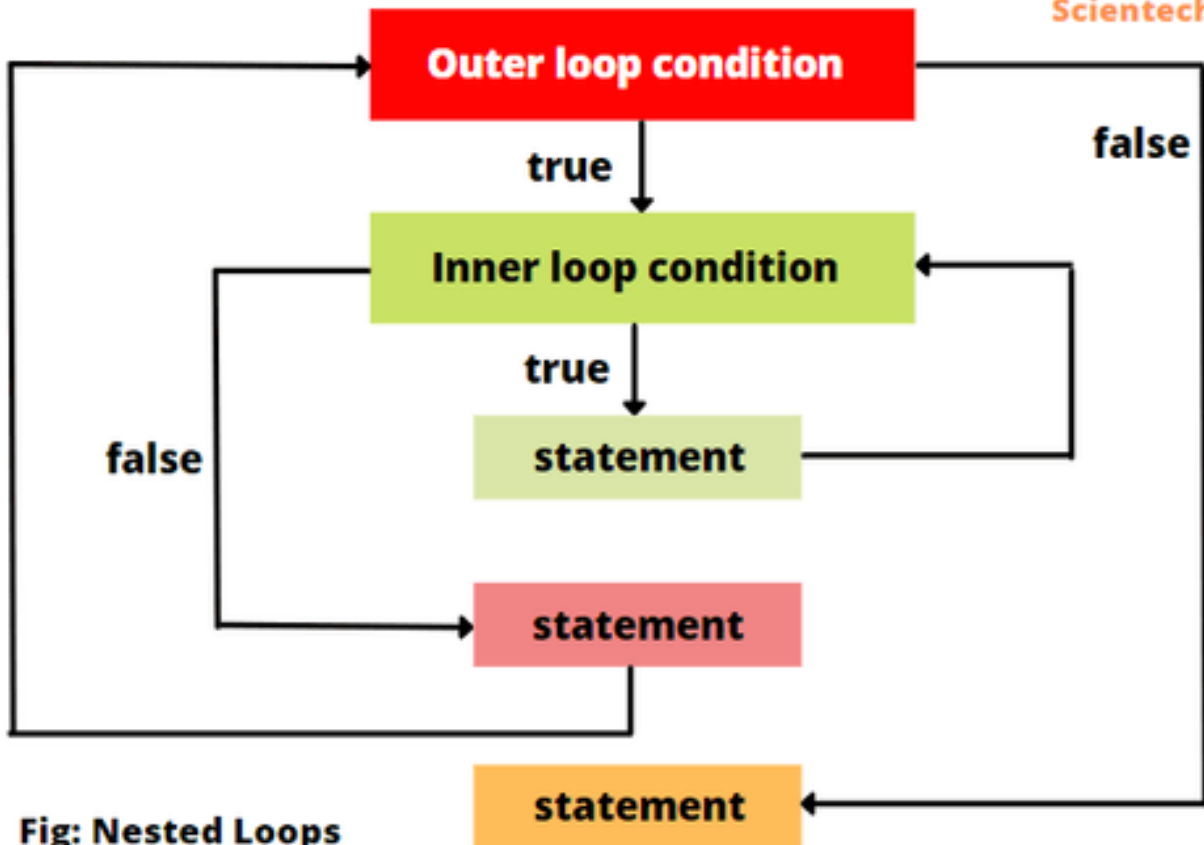


Fig: Nested Loops

Example:

```

for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        System.out.print(i + "," + j + " ");
    }
    System.out.println();
}

```

3.5 For-Each Loop

- A loop designed for traversing collections, arrays, etc.

Syntax:

```
for (type variable : array) {  
    // Code to execute for each element  
}
```

Example:

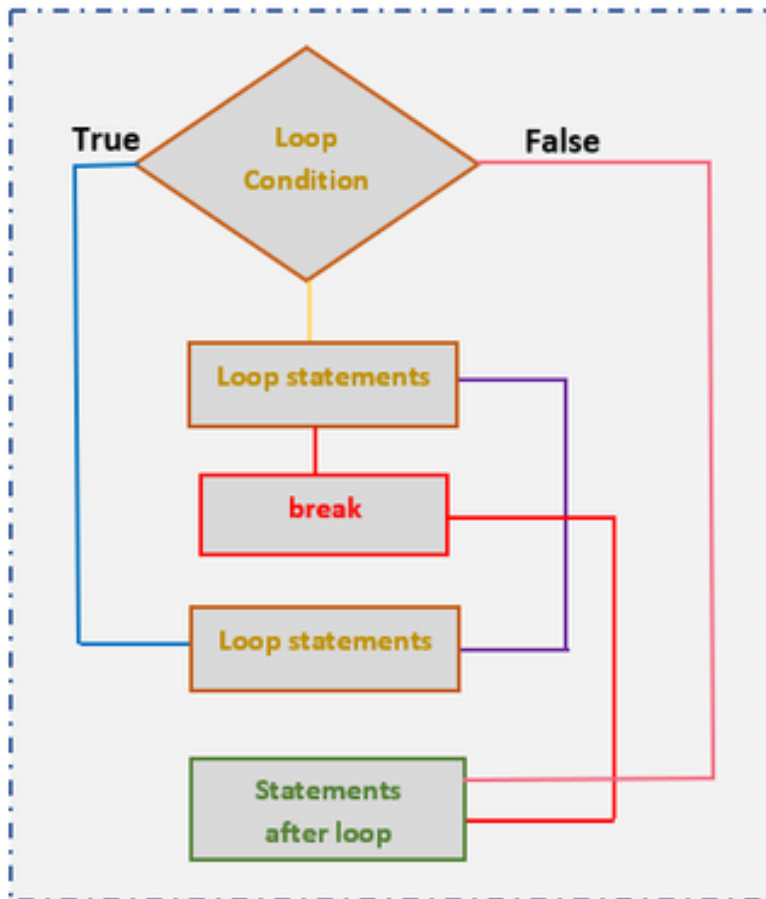
```
int[] numbers = {1, 2, 3, 4};  
  
for (int num : numbers) {  
    System.out.println(num);  
}
```

4. **Jump Statements**

4.1 **Break Statement**

- Used to exit a loop or switch prematurely.

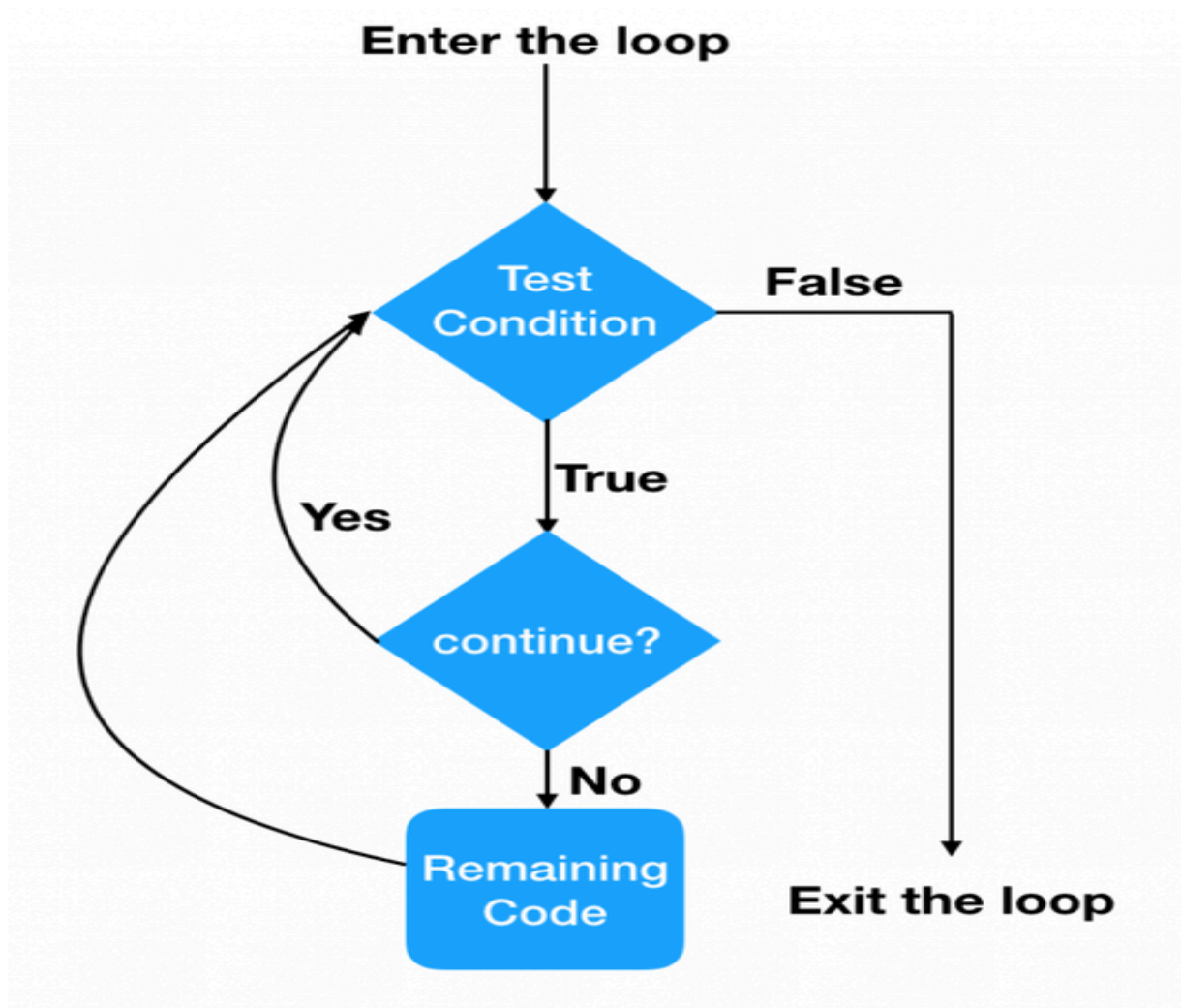
Example:



```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        break;  
    }  
    System.out.println(i);  
}
```

4.2 Continue Statement

- Skips the current iteration and proceeds to the next one.



Example:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue;  
    }  
    System.out.println(i);  
}
```